

The original **FL Language Manual** you can find at:

<https://theory.stanford.edu/~aiken/publications/trs/RJ7100.pdf>

(Functional Programming Group of IBM Research Division at Almaden USA 89)

Alberto Paoluzzi and his PLaSM-team

<https://web.archive.org/web/20200802000139/http://www.plasm.net/>

<https://plasmlanguage.org/apauluzzi/>

Tutorial

The design language PLaSM is a geometry-oriented extension of a subset of FL.

FL Language

FL (programming at Function Level) is an advanced language for functional programming developed by the **Functional Programming Group of IBM Research Division at Almaden (USA)** [BWW90,BWWLA89]. The FL language, on the line of the Backus' Turing lecture [Backus78] introduces an algebra over programs, i.e.~a set of algebraic identities between functional expressions, for formally reasoning about programs. This language enjoys some interesting features. In particular, programs are easily combined, so that new programs are obtained in a simple and elegant way; and one may find simpler equivalent programs, both at design and at compilation times. Great advantages are so obtained in style and efficiency of program prototyping.

PLaSM Language

PLaSM, (the Programming LAnguage for Solid Modeling) is a ``design language'', developed by the CAD Group at the Universities of Roma ``La Sapienza'' and ``Roma Tre'' [PS92, PPV95]. The language is strongly inFLuenced by FL. With few syntactical differences, it can be considered a geometric extension of a FL subset. Among the strong points of PLaSM we cite the functional approach, which allows to compute with geometries as well with numbers and functions, and its the dimension-independent implementation of geometric data structures and algorithms. The first feature results in a very natural and powerful approach to parametric geometry. The second one, coupled with the "combinatorial engine" of the FL language, gives an amazing descriptive power when computing with geometry.

Elements of FL syntax

FL is a pure functional language based on combinatorial logic.

The language introduces an algebra over programs (a set of algebraic identities between functional expressions) for reasoning formally about programs, so that one may find simpler equivalent programs (both at design and at compile times); programs are easily combined, so that new programs are obtained in a simple and elegant way; great advantages in style and efficiency of program development are achieved.

Primitive objects

characters, numbers and truth values

Expressions

primitive objects, functions, applications and sequences

Sequences

expressions separated by commas and contained within a pair of angle brackets: $\langle 5, \text{fun} \rangle$

The application expression

$\text{exp1}:\text{exp2}$ applies the function resulting from the evaluation of exp1 on the argument resulting from the evaluation of exp2 .

Binary functions

can also be used in infix form: $+: \langle 1, 3 \rangle = 1 + 3 = 4$

Application associates to left:

$f:g:h = (f:g):h$

Application binds stronger than composition:

$f:g \sim h = (f:g) \sim h$

FL combining forms and functions:

The construction combining form CONS

allows to apply a sequence of functions to an argument producing the sequence of applications:

$\text{CONS}:\langle f_1, \dots, f_n \rangle : x = [f_1, \dots, f_n]:x = \langle f_1:x, \dots, f_n:x \rangle$

E.g. $\text{CONS}:\langle +, - \rangle$, written also $[+, -]$, when applied to the argument $\langle 3, 2 \rangle$ gives the sequence of applications:

$[+, -]:\langle 3, 2 \rangle = \langle +:\langle 3, 2 \rangle, -:\langle 3, 2 \rangle \rangle = \langle 5, 1 \rangle$;

The apply-to-all combining form AA

applies a function to a sequence of arguments giving a sequence of applications:

$\text{AA}:f:\langle x_1, \dots, x_n \rangle = \langle f:x_1, \dots, f:x_n \rangle$;

The identity function ID

returns its argument unchanged: $\text{ID}:x = x$;

The constant function K

is evaluated, for whatever x_2 , as: $\text{K}:x_1:x_2 = x_1$;

Binary composition ~ of functions

is defined as $(f \sim g):x = f:(g:x)$;

N-ary composition COMP of functions

is also allowed: $\text{COMP}:< f,h,g >:x = (f \sim h \sim g):x = f:(h:(g:x))$;

The conditional form

IF:< p,f,g >:x is evaluated as follows:

IF:< p,f,g >:x = f:x if p:x = TRUE g:x if p:x = FALSE

The insert right and insert left combining forms (INSR and INSL)

allow to apply a binary function on a sequence of arguments of any length:

INSR:f:< x1,x2,...,xn > = f:< x1, INSR:f:< x2,...,xn > >

INSL:f:< x1,...,xn-1,xn > = f:< INSL:f:< x1,...,xn-1 >, xn >,

where $\text{INSR}:f:< x > = \text{INSL}:f:< x > = x$

The catenate function CAT

appends any number of input sequences creating a single output sequence:

CAT:<< a,b,c >, < d,e >, ..., < x,y,z >> = < a,b,c,d,e,...,x,y,z >

The distribute right and distribute left functions (DISTR, DISTL)

generate a sequence of pairs:

DISTR:<< a,b,c >, x> = << a,x >, < b,x >, < c,x >>

DISTL:< x,< a,b,c >> = << x,a >, < x,b >, < x,c >>

https://web.archive.org/web/20200802000224/http://www.plasm.net/docs/tutorial/elements_of_fl_syntax.html

The important Curry function:

This function embodies the fundamental idea of FL.

$C:f:x:y = f:<x,y>$

You can find it at:

<https://web.archive.org/web/20170402054505/http://plasm.net/Media/listings.png>

PLaSM Approach to Geometry

PLaSM, the Programming LAnguage for Symbolic Modeling, is a functional "design language" designed in recent years by the CAD Group at the University of Rome "La Sapienza" (Italy) and currently developed by the Geometric Computing Group at the University of Roma Tre.

PLaSM features

- Programming approach to "generative" modeling
- Geometrical objects generated by language expressions
- Algebraic calculus over embedded polyhedra
- Dim-independent approach to geometry repr and algorithms
- Extended variational geometry (classes of objects with varying topology and shape)
- Geometry-oriented extension of the functional language FL
- Validity of geometry syntactically guaranteed
- Domain broader than in solid modelers (points, wire-frames, surfaces, solids, etc.)

PLaSM is a geometry-oriented extension of a subset of FL:

- it is a functional "design language"
- can evaluate expressions whose value is a polyhedral complex
- can produce higher-level functions in the FL style
- no free nesting of scopes and environments is allowed
- no pattern matching is provided
- allows identifiers for any language object

The syntax of PLaSM is very similar to the one of FL. The differences only concern the meaning of few characters and the use of multiple lists of formal parameters of functions. Two new combining forms AC (apply-in-composition) and AS (apply-in-sequence) are introduced, in order to use a function with a number of actual parameters greater than the number used in the function definition.

Example

SEL is a primitive operator with a specification parameter i , such that when applied to a sequence the i -th sequence element is returned:

SEL:2:<13, 4.5, ID> = 4.5

By using the combining form AC and AS, the SEL operator can work in two different ways:

AC:SEL:< 3,1 >:<< 1,3,8,7 >, 89, fun>= (SEL:3 ~ SEL:1):<< 1,3,8,7 >, 89, fun> = 8

AS:SEL:< 3,1 >:<< 1,3,8,7 >, 89, fun>= [SEL:3, SEL:1]:<< 1,3,8,7 >, 89, fun>= < fun, < 1,3,8,7 >>

- PLaSM uses a case unsensitive alphabet;
- Allows for partially specified (curried) functions.
- Allows for overloading of operators:

3+2; 3*2

ops on numbers

(sin+cos):x; (sin*cos):x;

ops on functions

pol1+pol2; pol1*pol2;

ops on polyhedra

Predefined functions

Some non geometric predefined functions and combinatorial forms follow

Type predicates

(IsInt, IsReal, IsSeq, IsFun, IsPol)

Comparison functions

GT, GE, LT, LE, EQ, ...

Logical functions and constants

AND, OR, NOT, TRUE, <>, ...

Combining Forms

AS, AC, ...

Mathematics functions

SIN, COS, TAN, LN, EXP, ...

Sequence functions

AL, AR, FIRST, TAIL, CAT, ...

Some more non-geometric operators:

repetition operator allows for instancing n times any expression:

#:3:expr = < expr,expr,expr >

catenation of repeated sequences repeats and catenates sequences:

##:3:< a,b,c > = < a,b,c, a,b,c, a,b,c >

FROMTO operator generates integer sequences between two given extremes:

FromTo:<2,5> = 2..5 = <2,3,4,5>

INTSTO operator generates integer sequences with a given end:

INTSTO:5 = <1,2,3,4,5>

Function definition

Two kinds of functions are recognized in PLaSM:

global (or top-level) functions

Global functions may contain a definition of local functions between WHERE and END keywords. Global functions are allowed to contain formal parameters using a lambda-style. Formal parameters are specified together with a predicate which is normally used to perform some type-checking at run-time.

local functions

The visibility of local functions is restricted to the scope of a global function. Formal and actual parameters are given as follows:

Function definition:

```
DEF f (a::type1) = bodyF
DEF g (a1,...,an::type2) = bodyG
DEF h (a1::type1)(a2::type2) = bodyH           (Currying)
DEF l (a1::type1; a2::type2) = bodyL
```

The types are predicates like IsInt, IsReal, IsSeq, IsFun, etc

(Propositions as Types)

(Dependent Types)

Example at

https://web.archive.org/web/20170402054505/http://plasm.net/_Media/listings.png

Function instantiation:

```
f:x
g:< x1, ... , xn >
h:x1:x2
l:< x1, x2 >
```

[...]

https://web.archive.org/web/20200703133348/http://www.plasm.net/docs/tutorial/plasm_approach_to_geometry.html